

Product Requirements Document

Project Build 1: End-to-End Blog Generator Application Using Groq + Python

1. Project Title

AI-Powered Blog Generator for Marketing Teams

2. Project Duration

4 Hours

This is the first project build activity in the 8-hour project use-case day. Participants are expected to build a working prototype within 4 hours.

3. Project Context

A mid-sized technology company wants to improve the productivity of its marketing team by using Generative AI to create first drafts of high-quality blogs.

Currently, the marketing team manually creates blogs from rough product notes, campaign ideas, customer pain points, and target audience descriptions. This process is slow, inconsistent, and heavily dependent on individual writing skills.

The company wants an internal AI-powered blog generation tool that can take structured inputs from a marketing user and generate a complete blog package.

The tool should not simply generate random text. It should understand the business context, classify the blog intent, summarize the input, generate an outline, produce a polished blog, check quality, and return reusable marketing assets.

This project must be implemented using **Python** and **Groq API** for LLM access. Participants may optionally use a lightweight UI framework such as Streamlit, Gradio, Flask, or FastAPI.

4. Important Constraint

This project must be implemented **without RAG**.

Participants must not use:

- Vector databases
- Embedding search
- Document retrieval pipelines
- LangChain retrievers
- External knowledge base retrieval
- Web scraping
- Search APIs

The application should rely only on:

- User-provided inputs
 - Prompt engineering
 - Groq API calls
 - Python-based processing
 - Structured validation
 - Error handling
-

5. Learning Alignment

This project is mapped to the following topics covered in the training.

Training Topic	How It Is Applied in This Project
Introduction to AI & GenAI	Understanding where GenAI can assist marketing content creation
LLM Basics	Handling tokens, context window, hallucinations, and model output behavior
Prompt Engineering	Using zero-shot, one-shot, role prompting, and structured prompts
Advanced Prompting	Using few-shot examples, reasoning summaries, and hallucination control

Training Topic	How It Is Applied in This Project
Text Summarization	Summarizing product notes before blog generation
Text Classification	Classifying blog intent, audience, sentiment, and risk
API Integration	Calling Groq API from Python
Error Handling	Handling rate limits, retries, invalid JSON, timeout, and fallback behavior
Project Build	Building an end-to-end LLM-powered application

6. Business Problem

The marketing team needs a tool that can generate structured, brand-aligned blog content from limited business inputs.

The current manual process has the following problems:

1. Blog drafts take too long to prepare.
2. Different writers produce inconsistent tone and structure.
3. SEO elements are often missed.
4. Blogs sometimes contain vague or unsupported claims.
5. Marketing managers spend significant time reviewing and rewriting drafts.
6. Non-technical business teams cannot directly use LLM APIs.

The proposed application should reduce the effort required to create a first blog draft while still allowing human review before publishing.

7. Target Users

Primary User

Marketing Executive

A business user who wants to quickly generate a blog draft from rough notes.

Secondary User

Marketing Manager

A reviewer who checks whether the blog is aligned with campaign goals, target audience, brand tone, and quality expectations.

Optional User

Founder / Business Head

A stakeholder who wants quick thought-leadership content for LinkedIn, newsletters, or website blogs.

8. Product Goal

Build an AI-powered blog generator that takes structured user inputs and generates a complete blog package containing:

1. Blog intent classification
 2. Clean summary of user-provided notes
 3. Blog outline
 4. SEO-friendly title options
 5. Full blog draft
 6. Meta description
 7. LinkedIn post
 8. Quality review
 9. Hallucination risk checklist
 10. Final improvement suggestions
-

9. Sample Use Case

A SaaS company wants to publish a blog about how AI can improve customer support operations.

The marketing user provides:

- Topic: AI in customer support
- Target audience: Customer support leaders and operations heads
- Product/service: AI support assistant
- Key points:
 - Reduces repetitive ticket handling

- Summarizes customer issues
 - Suggests replies to support agents
 - Helps improve response time
 - Should not replace human agents completely
- Tone: Professional and practical
- Blog length: 900 words
- SEO keywords:
 - AI customer support
 - support automation
 - customer service AI
- Call to action:
 - Book a product demo

The system should generate a complete blog package based only on this information.

10. Functional Requirements

FR-1: User Input Collection

The application must accept the following inputs from the user:

Field	Required	Description
Blog Topic	Yes	Main topic of the blog
Target Audience	Yes	Intended readers
Product / Service Context	Yes	What the company offers
Key Points	Yes	Bullet points or rough notes
Desired Tone	Yes	Example: professional, persuasive, educational, conversational
Blog Length	Yes	Short, medium, long, or approximate word count
SEO Keywords	Yes	List of keywords to include
Call to Action	Yes	What the reader should

Field	Required	Description
Industry	Optional	do after reading Example: SaaS, healthcare, finance, education
Avoided Claims	Optional	Claims the model should not make
Brand Guidelines	Optional	Tone, style, vocabulary, or formatting preferences

FR-2: Input Validation

The application must validate user inputs before calling the Groq API.

Validation rules:

1. Blog topic cannot be empty.
2. Target audience cannot be empty.
3. Key points must contain at least 3 meaningful points.
4. Blog length must be one of the supported values.
5. SEO keywords must contain at least 2 keywords.
6. Call to action cannot be empty.

If validation fails, the application should return a clear error message.

Example:

Please provide at least 3 key points before generating the blog.

FR-3: Blog Intent Classification

Before generating the blog, the system must classify the blog intent.

Possible intent categories:

THOUGHT_LEADERSHIP
PRODUCT_EDUCATION
SEO_INFORMATIONAL
LEAD_GENERATION

COMPARISON
ANNOUNCEMENT
HOW_TO_GUIDE

The model should return:

```
{  
  "blog_intent": "LEAD_GENERATION",  
  "target_reader_maturity": "BEGINNER | INTERMEDIATE | ADVANCED",  
  "recommended_content_angle": "",  
  "reasoning_summary": ""  
}
```

The classification should help decide how the final blog should be written.

FR-4: Input Summarization

The application must summarize the user-provided notes before blog generation.

The summary should include:

```
{  
  "clean_summary": "",  
  "main_message": "",  
  "important_points": [],  
  "missing_information": [],  
  "possible_risks": []  
}
```

The summary should remove repetition, clarify vague inputs, and identify missing information.

FR-5: Blog Outline Generation

The application must generate a structured blog outline before generating the full blog.

The outline should include:

1. Suggested title
2. Introduction direction
3. 4 to 6 main sections
4. Key message for each section

5. Suggested conclusion
6. Call-to-action placement

Output format:

```
{
  "title": "",
  "outline": [
    {
      "section_heading": "",
      "section_purpose": "",
      "key_points_to_cover": []
    }
  ],
  "cta_placement": "",
  "estimated_word_count": 0
}
```

FR-6: Full Blog Generation

The application must generate a complete blog draft using the outline and summarized input.

The blog must include:

1. SEO-friendly title
2. Introduction
3. Body sections with headings
4. Practical examples or business context
5. Conclusion
6. Call to action

The blog must follow these rules:

- Use the selected tone.
- Include the given SEO keywords naturally.
- Avoid exaggerated or unsupported claims.
- Do not invent statistics.
- Do not mention customer names unless provided.
- Do not claim certifications, case studies, awards, or guarantees unless explicitly provided.
- Do not use RAG or external facts.

- Keep the writing professional and business-ready.
-

FR-7: SEO Metadata Generation

The application must generate:

```
{  
  "seo_title": "",  
  "meta_description": "",  
  "primary_keyword": "",  
  "secondary_keywords": [],  
  "suggested_slug": "",  
  "search_intent": ""  
}
```

Rules:

1. SEO title should be concise.
 2. Meta description should be under 160 characters.
 3. Keywords should be used naturally.
 4. Slug should be lowercase and hyphen-separated.
-

FR-8: LinkedIn Post Generation

The application must generate a LinkedIn post promoting the blog.

The LinkedIn post should include:

1. Hook
2. Short explanation
3. Business relevance
4. Call to action
5. 3 to 5 relevant hashtags

Output:

```
{  
  "linkedin_post": "",  
  "hashtags": []  
}
```

FR-9: Quality Review

The system must evaluate the generated blog using the LLM.

The review should score the blog on:

Criterion	Score Range
Relevance	1 to 5
Clarity	1 to 5
Structure	1 to 5
Tone Alignment	1 to 5
SEO Usage	1 to 5
Hallucination Risk	1 to 5
CTA Effectiveness	1 to 5

Output:

```
{
  "scores": {
    "relevance": 0,
    "clarity": 0,
    "structure": 0,
    "tone_alignment": 0,
    "seo_usage": 0,
    "hallucination_risk": 0,
    "cta_effectiveness": 0
  },
  "strengths": [],
  "improvement_areas": [],
  "final_quality_summary": ""
}
```

For hallucination risk, a higher score means higher risk. Participants should explain this clearly in their implementation.

FR-10: Hallucination Control Checklist

The application must identify statements in the blog that may require verification.

The model should extract:

```
{
  "claims_requiring_verification": [],

```

```
"unsupported_claims": [],  
"safe_claims": [],  
"recommended_edits": []  
}
```

The system should flag claims such as:

- Numerical claims
 - Market leadership claims
 - Guaranteed business outcomes
 - Legal or compliance claims
 - Customer success claims
 - Claims about competitors
 - Claims about certifications or awards
-

FR-11: Final Output Package

The application must produce a final structured response containing:

```
{  
  "blog_intent_analysis": {},  
  "input_summary": {},  
  "blog_outline": {},  
  "final_blog": "",  
  "seo_metadata": {},  
  "linkedin_post": {},  
  "quality_review": {},  
  "hallucination_check": {},  
  "generation_metadata": {  
    "model_used": "",  
    "temperature": 0,  
    "total_steps_completed": 0  
  }  
}
```

The final output should be displayed in the UI or printed clearly in the terminal.

11. Prompt Engineering Requirements

Participants must use at least the following prompt engineering techniques.

11.1 Role Prompting

The blog generation prompt must assign a clear role to the model.

Example:

You are a senior B2B content strategist and SEO blog writer.

The role should change depending on the task.

Examples:

Task	Suggested Role
Blog intent classification	B2B content strategist
Summarization	Business analyst
Blog outline	Content architect
Blog writing	Senior SEO blog writer
Quality review	Editorial reviewer
Hallucination check	Fact-checking assistant

11.2 Zero-Shot Prompting

Use zero-shot prompting for at least one of the following:

- Blog intent classification
- Hallucination checklist
- Quality review

The prompt should contain clear instructions but no examples.

11.3 One-Shot Prompting

Use one-shot prompting for at least one task.

Recommended use:

- SEO metadata generation
- LinkedIn post generation

The prompt should include one sample input-output example and then ask the model to process the actual user input.

11.4 Few-Shot Prompting

Use few-shot prompting for at least one task.

Recommended use:

- Blog outline generation
- Quality review
- Hallucination claim classification

The few-shot examples should teach the model how to handle:

1. Good claims
 2. Unsupported claims
 3. Overconfident claims
 4. Safe business statements
-

11.5 Reasoning Summary using but not limited to Tree of Thought, Chain of Thought

For analytical steps, ask the model to reason internally and return only a concise reasoning summary.

Do not ask the model to produce long hidden chain-of-thought.

Good instruction:

Think through the problem carefully, but return only a concise reasoning summary and the final structured answer.

11.6 Hallucination Control using but not limited to Self Consistency

All prompts must include clear rules such as:

Do not invent facts, statistics, customer names, awards, certifications, financial results, or legal claims.
If information is missing, state that it is not provided.

12. Technical Requirements

12.1 Recommended Technology Stack

Minimum required stack:

Python
Groq API
python-dotenv
JSON handling

Optional:

Streamlit
Gradio
FastAPI
Pydantic
Tenacity
Rich
Markdown

12.2 Suggested Folder Structure

Under project-build / llm-application directory

```
firstname-lastname/
├── app.py
├── groq_client.py
├── prompts.py
├── validators.py
├── output_parser.py
├── config.py
├── requirements.txt
├── .env.example
├── README.md
├── outputs/
│   └── sample_blog_output.json
└── tests/
    └── test_validators.py
```

Participants do not need to follow this exact structure, but their code should be organized.

12.3 Environment Variables

The API key should not be hard-coded.

The application should use:

GROQ_API_KEY=your_api_key_here

The .env.example file should contain:

```
GROQ_API_KEY=  
GROQ_MODEL=llama-3.3-70b-versatile
```

12.4 Groq API Integration

Participants must create a reusable function for calling the model.

Example function signature:

```
def call_groq_model(prompt: str, temperature: float = 0.3, max_tokens:  
int = 2000) -> str:  
    """  
    Calls Groq API and returns the model response as text.  
    """
```

The implementation should support:

1. Model name configuration
 2. Temperature configuration
 3. Timeout handling
 4. Retry handling
 5. Error messages
 6. JSON output parsing where required
-

12.5 Error Handling Requirements

The application must handle:

Error Type	Expected Handling
Missing API key	Show clear setup error
Empty user input	Show validation error
Invalid JSON from LLM	Retry once with JSON repair prompt
API timeout	Retry with exponential backoff
Rate limit	Wait and retry
Large input	Ask user to reduce input or summarize first
Model failure	Return graceful error message

13. Application Workflow

The application should follow this pipeline:

```
Step 1: Accept user inputs
Step 2: Validate inputs
Step 3: Classify blog intent
Step 4: Summarize input notes
Step 5: Generate blog outline
Step 6: Generate full blog
Step 7: Generate SEO metadata
Step 8: Generate LinkedIn post
Step 9: Review blog quality
Step 10: Run hallucination checklist
Step 11: Display final blog package
Step 12: Save output to JSON or Markdown
```

14. Input Example for Testing

Participants must test their application using the following sample input.

```
{
  "blog_topic": "How AI can improve customer support operations",
  "target_audience": "Customer support leaders, CX heads, and
operations managers",
  "product_or_service_context": "An AI-powered customer support
assistant that helps agents summarize tickets, draft responses,
identify escalation risks, and reduce repetitive manual work.",
  "key_points": [
    "Support teams are facing increasing ticket volumes.",
```



```
"Agents spend a lot of time reading long customer conversations.",
"AI can summarize tickets and suggest draft replies.",
"AI should assist human agents, not replace them.",
"The solution can help improve consistency and response speed.",
"Sensitive customer data must be handled carefully."
],
"desired_tone": "Professional, practical, and business-oriented",
"blog_length": "Medium, around 900 words",
"seo_keywords": [
  "AI customer support",
  "support automation",
  "customer service AI"
],
"call_to_action": "Book a demo to explore how AI can improve your
support operations.",
"industry": "SaaS",
"avoid_claims": [
  "Do not claim guaranteed cost reduction.",
  "Do not claim full automation of customer support.",
  "Do not mention any customer case study."
],
"brand_guidelines": "Use clear business language. Avoid hype. Keep
the tone trustworthy and practical."
}
```

15. Expected Output Characteristics

The final blog should:

1. Be relevant to the topic.
 2. Address the target audience clearly.
 3. Use the provided product context.
 4. Follow a professional structure.
 5. Include the SEO keywords naturally.
 6. Include the call to action.
 7. Avoid unsupported claims.
 8. Avoid unnecessary technical jargon.
 9. Include practical business examples.
 10. Be ready for human editorial review.
-

16. Non-Functional Requirements

NFR-1: Usability

The application should be easy for a non-technical marketing user to operate.

At minimum, the user should be able to:

- Enter inputs
 - Click or run a generate command
 - See the final blog package
 - Copy the generated blog
 - Review warnings and quality scores
-

NFR-2: Maintainability

Code should be modular.

Recommended separation:

Module	Responsibility
<code>groq_client.py</code>	API calls
<code>prompts.py</code>	Prompt templates
<code>validators.py</code>	Input validation
<code>output_parser.py</code>	JSON parsing
<code>app.py</code>	Application workflow

NFR-3: Reliability

The application should not crash when:

- The API fails
 - The model returns invalid JSON
 - User input is incomplete
 - The output is too long
 - The model refuses or gives incomplete response
-

NFR-4: Security

Participants must follow these security practices:

1. Do not hard-code API keys.
 2. Do not print API keys in logs.
 3. Do not commit .env files.
 4. Do not send unnecessary personal data to the model.
 5. Warn users not to enter confidential customer information.
-

NFR-5: Performance

The prototype should complete the full generation pipeline within a reasonable time.

Expected target:

Complete generation within 60 to 120 seconds for the sample input.

Participants may optimize by:

- Using fewer model calls
 - Reducing prompt length
 - Using structured outputs
 - Lowering max tokens for intermediate steps
-

17. Suggested Model Call Strategy

Participants may implement the application using 6 to 8 LLM calls.

Recommended approach:

Step	LLM Call Required
Intent classification	Yes
Input summarization	Yes
Blog outline	Yes
Blog draft	Yes
SEO metadata	Yes
LinkedIn post	Yes

Step	LLM Call Required
Quality review	Yes
Hallucination check	Yes

For time optimization, SEO metadata and LinkedIn post may be generated in a single call.

18. Prompt Design Expectations

Each prompt should contain:

1. Role
2. Task
3. Input context
4. Rules
5. Output format
6. Quality constraints

Example prompt structure:

Role:

You are a senior B2B content strategist.

Task:

Analyze the user-provided blog request and classify the blog intent.

Input:

{user_input}

Rules:

- Do not invent information.
- Base your answer only on the provided input.
- If information is missing, mention it.
- Return valid JSON only.

Output JSON:

```
{
  "blog_intent": "",
  "target_reader_maturity": "",
  "recommended_content_angle": "",
  "reasoning_summary": ""
}
```

19. Mandatory Features

Participants must implement the following features to pass the project:

Feature	Mandatory
Groq API integration	Yes
Input validation	Yes
At least 5 LLM-powered steps	Yes
Blog intent classification	Yes
Input summarization	Yes
Blog outline generation	Yes
Full blog generation	Yes
Quality review	Yes
Hallucination checklist	Yes
JSON or Markdown export	Yes
Error handling	Yes

20. Optional Advanced Features

Participants may add the following if time permits:

1. Streamlit or Gradio UI
 2. Download blog as Markdown
 3. Multiple blog title variations
 4. Tone selector
 5. Persona selector
 6. Retry button
 7. Blog regeneration with feedback
 8. Before-and-after prompt comparison
 9. Token estimation
 10. Logging of model calls
 11. Simple unit tests for validators
 12. Cost or call count tracker
-

21. Evaluation Rubric

Total Marks: **100**

Evaluation Area	Marks
Functional completeness	20
Prompt engineering quality	20
Groq API integration	15
Output structure and parsing	10
Hallucination control	10
Error handling and reliability	10
Code quality and modularity	10
Final analysis and documentation	5

22. Detailed Scoring Criteria

22.1 Functional Completeness — 20 Marks

Criteria	Marks
Accepts required user inputs	3
Performs blog intent classification	3
Generates input summary	3
Generates structured outline	3
Generates final blog	4
Generates SEO and LinkedIn assets	2
Performs review and hallucination check	2

22.2 Prompt Engineering Quality — 20 Marks

Criteria	Marks
Uses role prompting correctly	3

Criteria	Marks
Uses zero-shot prompting correctly	3
Uses one-shot or few-shot prompting correctly	4
Uses clear output schemas	3
Includes hallucination control rules	3
Uses concise reasoning summary where required	2
Produces specific, non-generic outputs	2

22.3 Groq API Integration — 15 Marks

Criteria	Marks
Correct Groq client setup	4
API key handled securely	3
Reusable API function	3
Configurable model and temperature	2
Handles API errors gracefully	3

22.4 Output Structure and Parsing — 10 Marks

Criteria	Marks
Produces structured JSON outputs	3
Parses model responses correctly	3
Handles invalid JSON	2
Saves final output to file	2

22.5 Hallucination Control — 10 Marks

Criteria	Marks
Explicitly instructs model not to invent facts	2
Flags unsupported claims	3
Identifies claims requiring verification	2
Uses provided context only	2
Provides recommended edits	1

22.6 Error Handling and Reliability — 10 Marks

Criteria	Marks
Handles missing inputs	2
Handles missing API key	2
Handles API timeout or failure	2
Handles rate limit or retry scenario	2
Gives user-friendly error messages	2

22.7 Code Quality and Modularity — 10 Marks

Criteria	Marks
Clean folder structure	2
Modular functions	3
Clear variable names	2
Minimal duplicate code	1
Readable README	2

22.8 Final Analysis and Documentation — 5 Marks

Participants must include a short README.md or project notes file explaining:

1. How to run the application
 2. Which prompt engineering techniques were used
 3. How hallucination was controlled
 4. Known limitations
 5. Future improvements
-

23. Acceptance Criteria

The project will be considered successful if:

1. The application runs without crashing.
 2. The user can provide blog requirements.
 3. The system calls Groq API successfully.
 4. The system generates a complete blog package.
 5. The output includes classification, summary, outline, blog, SEO metadata, LinkedIn post, quality review, and hallucination check.
 6. The blog is based only on user-provided information.
 7. The application includes basic error handling.
 8. The final output is saved as JSON or Markdown.
 9. The participant can explain the prompt engineering choices used.
-

24. Sample Expected Final Output Structure

```
{
  "blog_intent_analysis": {
    "blog_intent": "LEAD_GENERATION",
    "target_reader_maturity": "INTERMEDIATE",
    "recommended_content_angle": "Position AI as an assistant that improves support team productivity without replacing human agents.",
    "reasoning_summary": "The input focuses on operational improvement, support efficiency, and a demo CTA, making lead generation the strongest intent."
  },
  "input_summary": {
    "clean_summary": "The blog should explain how AI can help support teams handle increasing ticket volumes by summarizing conversations, drafting replies, and improving consistency while keeping humans involved.",
    "main_message": "AI can support customer service teams by reducing repetitive effort and improving response quality.",
    "important_points": [
```

```

        "Support teams face rising ticket volumes.",
        "Agents spend time reading long conversations.",
        "AI can summarize tickets and suggest responses.",
        "AI should assist rather than replace agents.",
        "Sensitive customer data must be handled carefully."
    ],
    "missing_information": [
        "No specific product metrics are provided.",
        "No customer case studies are provided."
    ],
    "possible_risks": [
        "The blog should avoid claiming guaranteed cost reduction.",
        "The blog should avoid implying full automation."
    ]
},
"blog_outline": {},
"final_blog": "",
"seo_metadata": {},
"linkedin_post": {},
"quality_review": {},
"hallucination_check": {},
"generation_metadata": {
    "model_used": "llama-3.3-70b-versatile",
    "temperature": 0.3,
    "total_steps_completed": 8
}
}

```

25. Submission Requirements

Participants must submit:

1. Source code
2. requirements.txt
3. .env.example
4. README.md
5. Sample output file
6. Short explanation of prompt engineering techniques used

Suggested final folder:

Under project-build / llm-application

```

firstname-lastname/
├── app.py
├── groq_client.py
└── prompts.py

```

```
├── validators.py
├── output_parser.py
├── requirements.txt
├── .env.example
├── README.md
├── outputs/
│   └── sample_blog_output.json
```

26. Restrictions

Participants must not:

1. Use RAG.
 2. Use vector databases.
 3. Use external web search.
 4. Hard-code the final blog.
 5. Hard-code API outputs.
 6. Store API keys in source code.
 7. Ignore validation.
 8. Submit only a prompt without a working application.
 9. Generate unsupported claims or fake statistics.
 10. Use copied outputs without explaining their implementation.
-

27. Complexity Expectations

This is not a simple “generate a blog” task.

A strong submission should demonstrate:

1. Multiple LLM calls arranged in a logical pipeline.
2. Clear prompt engineering.
3. Structured outputs.
4. Validation and error handling.
5. Quality review.
6. Hallucination control.
7. Business-ready final output.

8. Clear explanation of implementation choices.

28. Final Instruction to Participants

You are building a realistic AI content generation assistant for a marketing team.

Focus on:

- Correct use of Groq API
- Good prompt design
- Clean Python implementation
- Structured outputs
- Business usefulness
- Hallucination control
- Error handling

Do not aim to create the longest blog. Aim to create a reliable, controlled, and useful GenAI application that a real marketing team could use as a first-draft assistant.